

2025-2

Modelos y Servicios de Datos (MYSD)

Clase #8: Integridad declarativa



UNIVERSIDAD

Quiz #1 - Repaso clase anterior (Kahoot)

- En parejas
- Unirse con sus apellidos. Ej:
 - Cardenas-Carmona

Mejor quiz +0,1 en el parcial 2T

AyalaCastelblanco

Quiz - Control de lectura

Mejor quiz +0,1 en el parcial 2T

1. ¿Qué significa integridad de los datos? Dé un ejemplo de la vida real.
1. ¿Cuál es la diferencia entre integridad y seguridad de datos?
1. Mencione y explique al menos 3 mecanismos de restricciones de integridad en SQL. Dé un ejemplo de cada uno.

Modelo relacional

Aspectos basicos

- ▶ **Aspecto estructural**

Los usuarios perciben la información de la base de datos como tablas y nada más que tablas

- ▶ **Aspecto de integridad**

Estas tablas satisfacen ciertas restricciones de integridad

- ▶ **Aspecto de manipulación**

Las operaciones disponibles permiten derivar tablas a partir de tablas

Integridad declarativa

Es el Black Friday en Mercado Libre

¿Qué pasaría si?:

- Un cliente se registra con un email que ya está en uso.
- Un cliente compra 5 unidades de un producto del que solo quedan 2 en stock.



Integridad declarativa

Imaginen que la base de datos es como la construcción de un edificio.

Necesitamos reglas para que sea sólido y no se caiga.

Algunas **reglas** son como los planos: declaran cómo debe ser la estructura

- Una pared no puede estar en medio de una puerta.
- El edificio no puede tener más de 10 pisos.



Integridad declarativa

- La integridad declarativa se enfoca en definir las **reglas** que los datos deben cumplir.
- Le decimos a la base de datos **qué es válido**, y ella se encarga automáticamente de hacerlo cumplir.
- Es la primera línea de defensa para la **calidad** de los datos.



Integridad declarativa

Mecanismos simplificados

1. Tablas: Tipo de dato, Nulidad
2. Claves: Primarias - Foráneas - Únicas

Mecanismos generales

1. Definición de dominio (De atributo)
2. Condición de tupla (De atributo De tupla)
3. Aserción general (De tabla De base de datos)

Definición de dominio (nivel de atributo)

¿Qué es?

- Es la forma más básica de integridad.
- Se asegura de que los valores en una columna sean del tipo correcto y estén dentro de un rango o conjunto de valores permitidos.
- Se aplica a **UNA SOLA** columna/atributo

¿Para qué lo hacemos?

- Para evitar datos sin sentido, como un texto en un campo de fecha o un precio negativo.

```
CREATE DOMAIN TColor AS VARCHAR(10)
CONSTRAINT ck_color CHECK (VALUE IN ('Rojo','Amarillo','Verde'));
```

Definición de dominio (nivel de atributo)

Mecanismos:

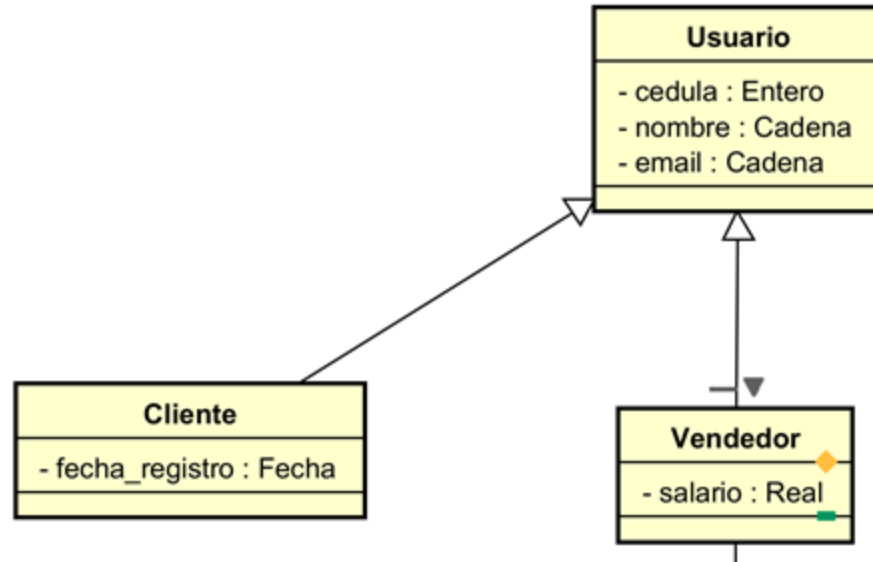
- Tipos de datos:
 - INT, VARCHAR, DECIMAL, DATE, etc.
- Restricciones de columna:
 - NOT NULL, UNIQUE, PRIMARY KEY, CHECK.
- CREATE DOMAIN:
 - Un mecanismo estándar de SQL (aunque no todos los motores lo implementan igual) para crear un tipo de dato personalizado con sus propias reglas.

```
CREATE DOMAIN TColor AS VARCHAR(10)
CONSTRAINT ck_color CHECK (VALUE IN ('Rojo','Amarillo','Verde'));
```

Definición de dominio (nivel de atributo)

Ejercicio:

- Definir el dominio en SQL para esta herencia:



Condición de tupla (De atributo/De tupla)

¿Qué es?

- **Pueden** involucrar la relación entre **dos o más columnas** dentro de esa misma fila (si son de tupla).
- Se usa con **CHECK**

¿Para qué lo hacemos?

- Para validar condiciones de negocio que dependen de múltiples atributos.
- Por ejemplo, que la "fecha de envío" sea siempre posterior a la "fecha de compra".

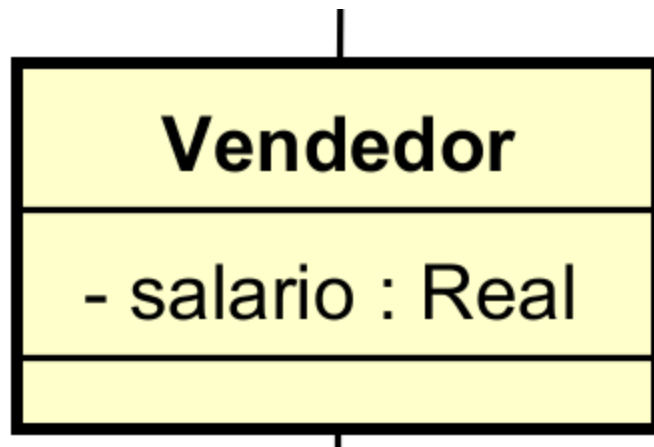
```
ALTER TABLE vendedores  
ADD CONSTRAINT chk_salario_positivo  
CHECK (salario > 0);
```

¿Este código es
de atributo o de
tupla?

Condición de tupla (De atributo/De tupla)

Ejercicio:

- Definir la siguiente condición de tupla en SQL:
 - El salario de un vendedor NO puede ser negativo.



Condición de tupla (De atributo/De tupla)

Ejemplo

- El precio del descuento nunca puede ser un precio negativo y que el precio con descuento sea menor al original.

¿Este código es
de atributo o de
tupla?

Producto
- id : Entero
- nombre : Cadena
- precio : Decimal
- precio_descuento : Decimal

```
ALTER TABLE Productos
ADD CONSTRAINT chk_precio_valido
CHECK (precio_descuento < precio AND precio_descuento ≥ 0);
```

Aserción general (De tabla/De base de datos)

¿Qué es una aserción?

```
def test_greet():  
    bob = {'name': "Bob"}           # Arrange  
    greeting = greet(bob)           # Act  
    assert greeting == "Hi Bob"     # Assert
```

¿Código de
greet(name)?

```
public class CalculatorTest {  
  
    @Test  
    public void testSumOfTwoNumbers() {  
        // Arrange  
        Calculator calculator = new Calculator();  
        int first = 5;  
        int second = 10;  
  
        // Act  
        int sum = calculator.sum(first, second);  
  
        // Assert  
        Assert.assertEquals( expected: 15, sum);  
    }  
}
```

¿Código de sum(first, second)?

Aserción general (De tabla/De base de datos)

¿Qué es?

- Son reglas de negocio complejas que pueden involucrar múltiples tablas, filas y agregaciones (SUM, COUNT, etc.).
- Una aserción es una regla global que la base de datos debe verificar siempre que se produzca un cambio que pueda afectarla.
- Se usa el **NOT EXISTS**

¿Para qué lo hacemos?

- Para hacer cumplir políticas globales y **reglas de negocio** de la empresa.

Aserción general (De tabla/De base de datos)

Ejemplo

- Un cliente no puede tener más de 5 pedidos con un total superior a \$1000 en el mismo mes

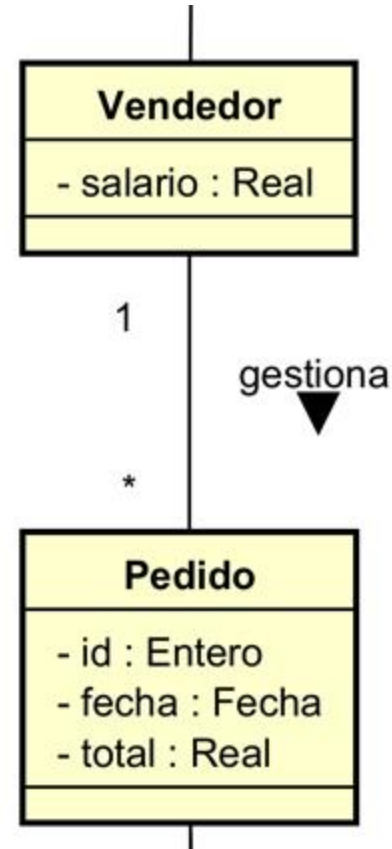
```
CREATE ASSERTION limite_pedidos_grandes_cliente
CHECK (NOT EXISTS (
    SELECT c.id, COUNT(p.id)
    FROM clientes c
    JOIN usuarios u ON c.id = u.id
    JOIN pedidos p ON u.id = p.cliente_id
    WHERE p.total > 1000
    GROUP BY c.id, TO_CHAR(p.fecha, 'YYYY-MM')
    HAVING COUNT(p.id) > 5
));
```

Aserción general (De tabla/De base de datos)

Ejercicio: Mejor grupo +0,1 en el parcial 1T
Palomares, Patiño, Sua, Munar

- Definir la siguiente aserción en SQL:
 - Garantizar que un vendedor no tenga más de 100 pedidos abiertos.

```
CREATE ASSERTION max_pedidos_vendedor
CHECK (NOT EXISTS (
    SELECT vendedor
    FROM pedidos
    GROUP BY vendedor
    HAVING COUNT(*) > 100
));
```



Ejercicios - STOP

- Grupos de 4
- El primer grupo que termine cada ejercicio deberá gritar “STOP”.
- Todos los demás grupos deben detenerse y escuchar la solución propuesta.
- Se escogerá un miembro del grupo al azar para que sustente su solución.
- Si tienen TODO ok = +1 punto
- Si tienen AL MENOS un error = -2 puntos
- Deben seguir todas las BUENAS PRÁCTICAS, de lo contrario se considerará como ERROR.
- Gana el grupo que tenga más puntos al final
- **Mejor grupo +0.5 en el parcial 2T**
- **Rojas, Rico, Novoa, Barrera**

Ejercicios - STOP

Necesitamos asegurarnos de que, por ningún motivo, un producto en nuestro catálogo pueda tener un precio de cero o un valor negativo. Cualquier precio que se registre debe ser mayor que cero.

Producto
- id : Entero - nombre : Cadena - precio : Decimal - precio_descuento : Decimal

```
ALTER TABLE Productos  
ADD CONSTRAINT chk_precio_positivo CHECK (precio > 0);
```

Ejercicios - STOP

Todo producto debe costar al menos 100 unidades monetarias.

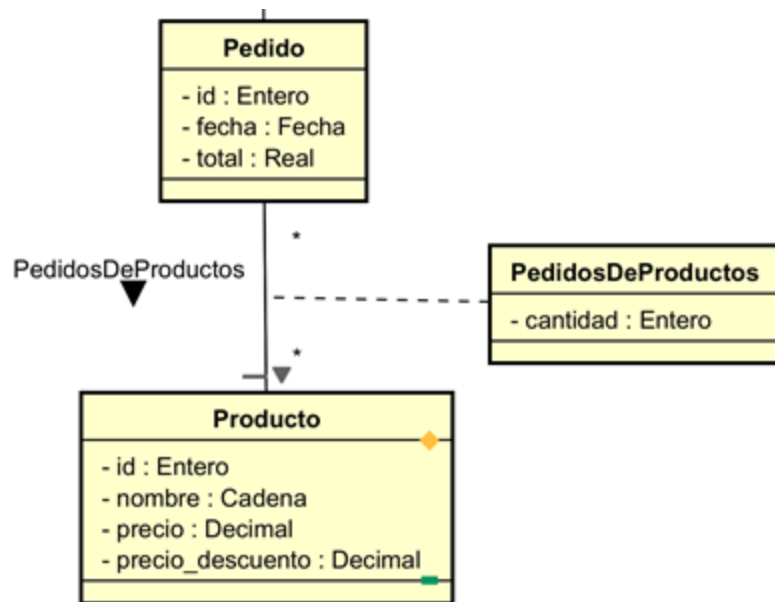
```
ALTER TABLE Productos  
ADD CONSTRAINT ck_precio_min  
CHECK (precio  $\geq$  100);
```

Producto
- id : Entero - nombre : Cadena - precio : Decimal - precio_descuento : Decimal

Ejercicios - STOP

Cuando un cliente añade un producto a su pedido, no tiene sentido que la cantidad sea cero o un número negativo. Implementar una regla para que la cantidad de cualquier artículo en un pedido sea siempre de 1 en adelante.

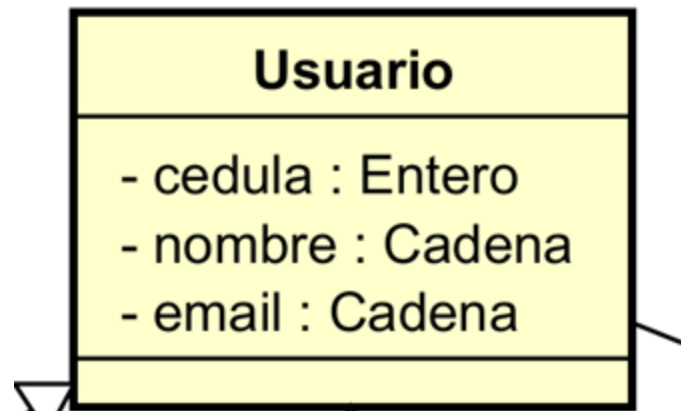
```
ALTER TABLE PedidosDeProductos  
ADD CONSTRAINT chk_cantidad_minima  
CHECK (cantidad ≥ 1);
```



Ejercicios - STOP

Estamos viendo usuarios que se registran con correos que no son válidos, como 'juan123' o 'info@local'. Necesitamos una validación muy simple que obligue a que todos los correos electrónicos tengan al menos una arroba ('@') y un punto('.') después de la arroba.

```
ALTER TABLE Usuarios  
ADD CONSTRAINT chk_formato_email  
CHECK (email LIKE '%_@__%.__%');
```

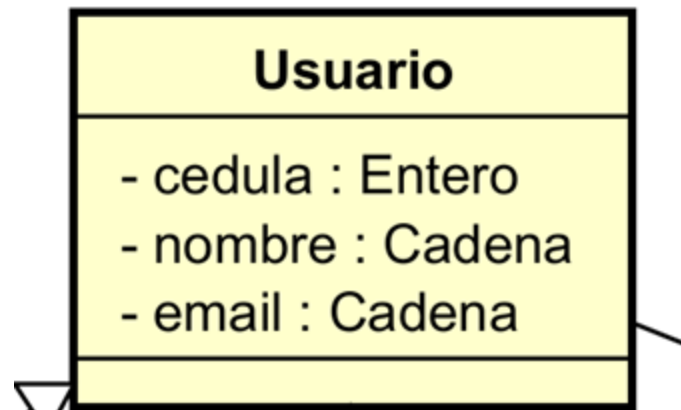


Ejercicios - STOP

Ningún usuario puede quedar sin nombre y la cantidad máxima es de 50 caracteres.

```
CREATE TABLE Usuarios(  
  nombre VARCHAR(50) NOT NULL  
  ... );
```

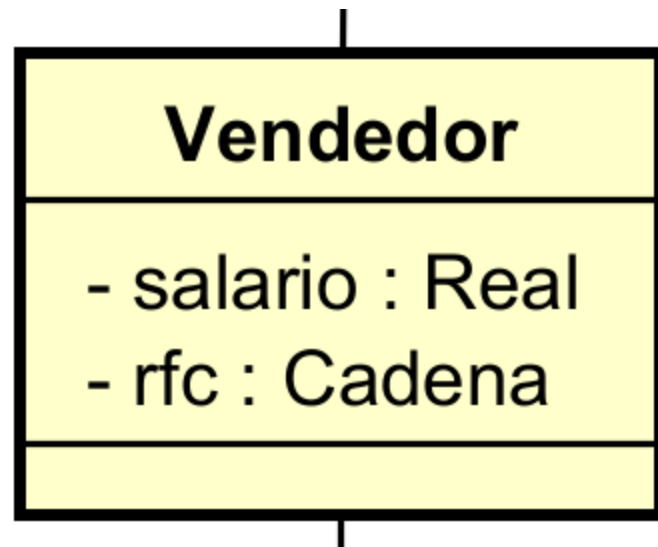
```
ALTER TABLE Usuarios  
MODIFY nombre VARCHAR(50) NOT NULL;
```



Ejercicios - STOP

El RFC de un vendedor debe tener exactamente 13 caracteres.

```
ALTER TABLE Vendedores  
ADD CONSTRAINT ck_rfc_len  
CHECK (LENGTH(rfc) = 13);
```



Ejercicios - STOP

El precio de venta al público siempre debe ser mayor que el costo. Nunca podemos vender algo por menos de lo que nos costó.

```
ALTER TABLE productos  
ADD CONSTRAINT chk_margen_ganancia  
CHECK (precio > precio_costo);
```

Producto
- id : Entero - nombre : Cadena - precio : Decimal - precio_descuento : Decimal - precio_costo : Decimal

Ejercicios - STOP

El tipo de usuario solo puede ser “Cliente” o “Vendedor”.

```
ALTER TABLE Usuarios  
ADD CONSTRAINT ck_tipo_usuario  
CHECK (tipo IN ('Cliente', 'Vendedor'));
```

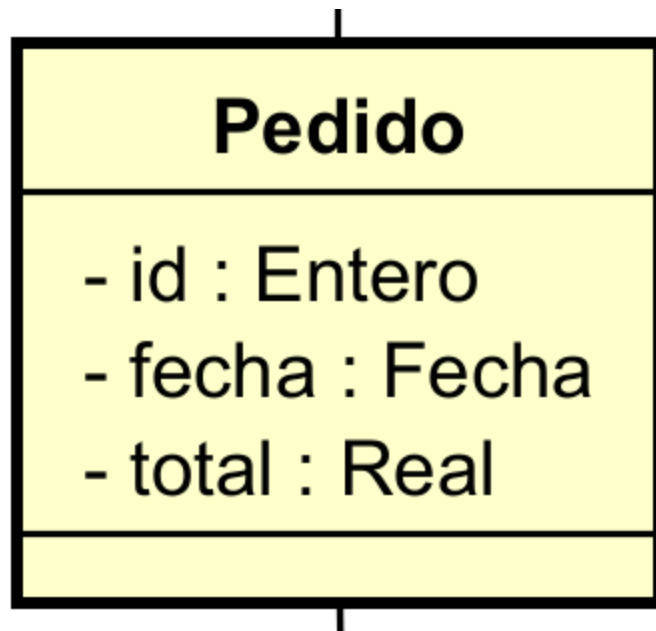
Usuario

- cedula : Entero
- nombre : Cadena
- email : Cadena
- tipo : Cadena

Ejercicios - STOP

Todo pedido debe tener un total positivo y una fecha que no esté en el futuro.

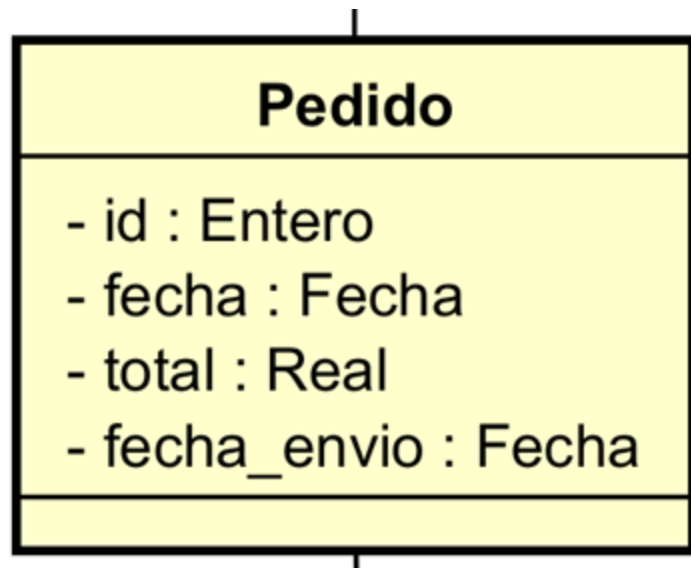
```
ALTER TABLE Pedidos  
ADD CONSTRAINT ck_total_fecha  
CHECK (total > 0 AND fecha ≤ CURRENT_DATE);
```



Ejercicios - STOP

Es imposible registrar una fecha de envío que sea anterior a la fecha en que se creó el pedido. Un paquete no se puede enviar antes de ser comprado.

```
ALTER TABLE Pedidos  
ADD CONSTRAINT chk_secuencia_fechas  
CHECK (fecha_envio ≥ fecha);
```



Ejercicios - STOP

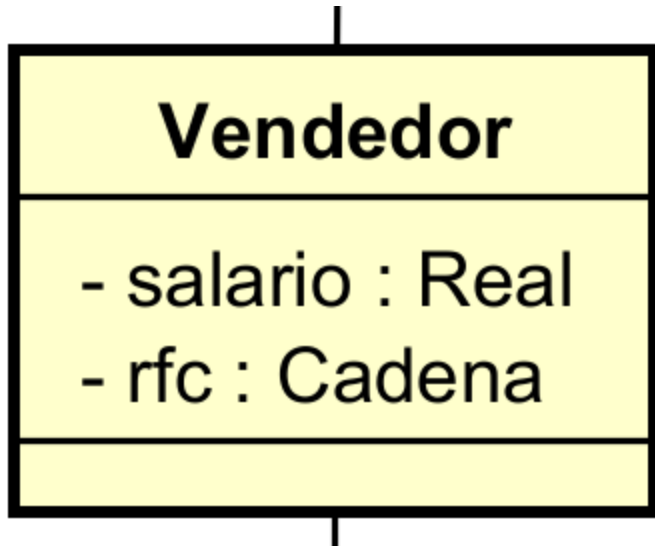
PUNTOS DOBLES

Si un vendedor gana más de 5000, su RFC debe empezar con la letra "A".

PISTA:

1. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE Vendedores  
ADD CONSTRAINT ck_salario_rfc  
CHECK (salario > 5000 OR rfc LIKE 'A%');
```



Ejercicios - STOP

PUNTOS DOBLES

Si un usuario es cliente, su correo debe terminar en “@gmail.com”.

PISTA:

1. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE usuarios
ADD CONSTRAINT ck_cliente_gmail
CHECK (tipo <> 'cliente'
      OR email LIKE '%@gmail.com');
```

Usuario

- cedula : Entero
- nombre : Cadena
- email : Cadena
- tipo : Cadena

Ejercicios - STOP

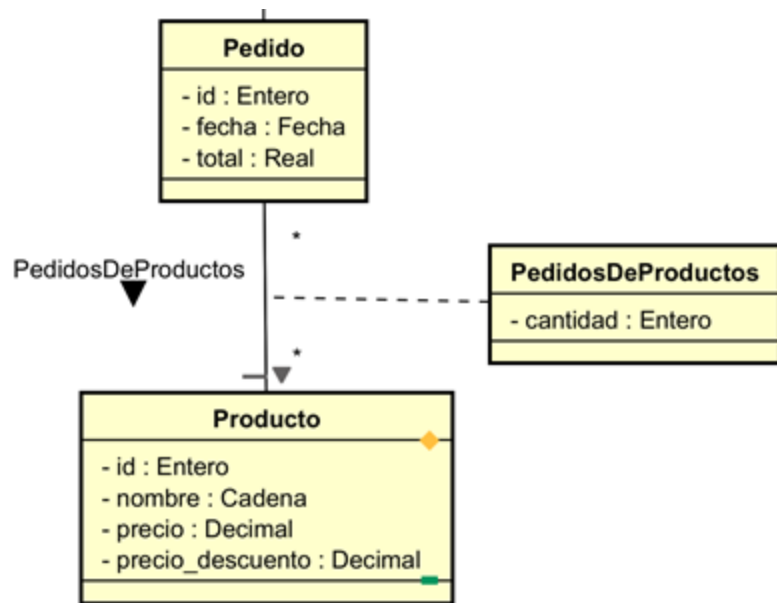
PUNTOS TRIPLES

Si un pedido de un producto tiene una cantidad mayor a 50, debe corresponder a un pedido con total mayor o igual a 5000.

PISTAS:

1. Hacer modelo lógico
2. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE PedidosDeProductos
ADD CONSTRAINT ck_pedido_grande
CHECK (
    cantidad ≤ 50 OR
    id_pedido IN (SELECT id
                  FROM Pedidos
                  WHERE total ≥ 5000)
);
```



Ejercicios - STOP

PUNTOS DOBLES

Si un usuario es cliente, su correo debe terminar en “@gmail.com”.

PISTA:

1. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE Usuarios
ADD CONSTRAINT ck_cliente_gmail
CHECK (tipo <> 'cliente'
      OR email LIKE '%@gmail.com');
```

Usuario

- cedula : Entero
- nombre : Cadena
- email : Cadena
- tipo : Cadena

Ejercicios - STOP

PUNTOS DOBLES

Si la ciudad es Bogotá, el código postal debe empezar por “11”.

PISTA:

1. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE Direcciones
ADD CONSTRAINT ck_cod_bogota
CHECK (ciudad <> 'Bogotá'
      OR codigo_postal LIKE '11%');
```

Dirección
- id : Entero - calle : Cadena - ciudad : Cadena - codigo_postal : Cadena

Ejercicios - STOP

PUNTOS DOBLES

Si un producto se llama “Laptop”, debe costar más de 2000.

PISTA:

1. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE Direcciones
ADD CONSTRAINT ck_cod_bogota
CHECK (ciudad <> 'Bogotá'
      OR codigo_postal LIKE '11%');
```

Producto
- id : Entero
- nombre : Cadena
- precio : Decimal
- precio_descuento : Decimal
- precio_costo : Decimal

Ejercicios - STOP

PUNTOS TRIPLES

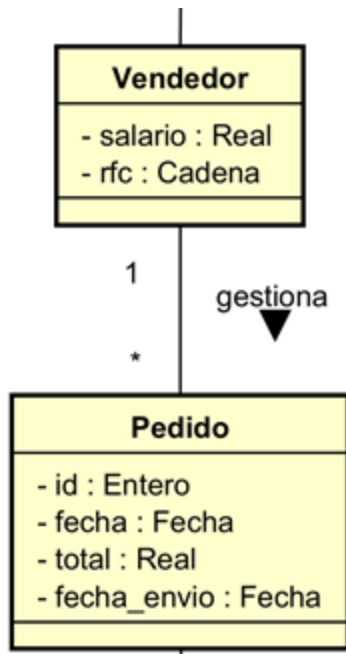
Si un vendedor gana más de \$8000, entonces debe tener al menos 5 pedidos asociados. Si gana menos o igual a \$8000, puede tener cualquier cantidad de pedidos.

PISTAS:

1. Hacer modelo lógico
2. $(P \rightarrow Q) = (\neg P \vee Q)$

```
ALTER TABLE Vendedores
ADD CONSTRAINT ck_vendedor_pedidos
CHECK (
    salario ≤ 8000
    OR id IN (
        SELECT vendedor
        FROM pedidos
        GROUP BY vendedor
        HAVING COUNT(*) ≥ 5
    )
);
```

```
ALTER TABLE Vendedores
ADD CONSTRAINT ck_vendedor_pedidos
CHECK (
    salario ≤ 8000
    OR (
        SELECT COUNT(*)
        FROM pedidos p
        WHERE p.vendedor = vendedores.id
    ) ≥ 5
);
```



Thanks ;)